

1) Circular Queue using array (maxsize = 6)

* Queue Size = 6

* Initially : Front = 0, Rear = -1, Count = 0

Step-by-step operations

Step	operation	Queue (0-5)	Front	Rear	Count
a	Initially Empty	[-, -, -, -, -, -]	0	-1	0
b	Enqueue A, B, C	[A, B, C, -, -, -]	0	2	3
c	Dequeue A	[-, B, C, -, -, -]	1	2	2
d	Enqueue D, E, F	[-, B, C, D, E, F]	1	5	5
e	Dequeue B, C	[-, -, -, D, E, F]	3	5	3
f	Enqueue G	[G, -, -, D, E, F]	3	0	4
g	Dequeue D, E	[G, -, -, -, -, F]	5	0	2
h	Enqueue H, I	[G, H, I, -, -, F]	5	2	4
i	Dequeue F	[G, H, I, -, -, -]	0	2	3
j	Dequeue G, H	[-, -, I, -, -, -]	2	2	1
k	Dequeue I	[-, -, -, -, -, -]	3	2	0

Circular Queue Sketch

(a) Initially Empty

Index : 0 1 2 3 4 5
 - - - - - -

Front = 0

Rear = -1

Count = 0

(b) Enqueue A, B, C

Index :	0	1	2	3	4	5
	A	B	C	-	-	-

F $\rightarrow$ 0

R $\rightarrow$ 2

Count = 3

(c) Dequeue A

Index :	0	1	2	3	4	5
	-	B	C	-	-	-

F $\rightarrow$ 1

R $\rightarrow$ 2

Count = 2

(d) enqueue D, E, F

Index :	0	1	2	3	4	5
	-	B	C	D	E	F

F $\rightarrow$ 1

R $\rightarrow$ 5

Count = 5

(e) dequeue B, C

Index :	0	1	2	3	4	5
	-	-	-	D	E	F

F $\rightarrow$ 3

R $\rightarrow$ 5 ; Count = 3

(f) enqueue G (wrap Around)

Index : 0 1 2 3 4 5
G - - D E F

F $\rightarrow$ 3

R $\rightarrow$ 0

Count = 4

(g) Dequeue D, E

Index : 0 1 2 3 4 5
G - - - - F

F $\rightarrow$ 5

R $\rightarrow$ 0

Count = 2

(h) Enqueue H, I

Index : 0 1 2 3 4 5
G H I - - F

F $\rightarrow$ 5

R $\rightarrow$ 2

Count = 4

(i) Dequeue F

Index : 0 1 2 3 4 5
G H I - - -

F $\rightarrow$ 0

R $\rightarrow$ 2

Count = 3

(j) Dequeue C14

Index : 0 1 2 3 4 5
- - - - -

$F \rightarrow 2$

$R \rightarrow 2$

Count = 1

(k) Dequeue 1 (Queue Empty)

Index : 0 1 2 3 4 5
- - - - -

Front = 3

Rear = 2

Count = 0

Final observation

Initial :

Front = 0, Rear = -1, Count = 0.

After all operations : Front = 3, Rear = 2, Count = 0

(Queue empty).

The circular queue efficiently reuses empty locations at the beginning of the array after the rear reaches the last index, avoiding wasted space unlike a linear queue.

Linked List Configuration using list ADT

Assumption:

- * append(x) adds the element at the end of the list.
- * insert(x) inserts the element at the current position.
- * moveToStart() moves the current position to the first node.
- * next() moves the current position to the next node.
- * The symbol $\downarrow$ indicates the current position (cursor).

(a) operations

L1. append(10);

L1. append(20);

L1. append(15);

Step 1

After:

L1. append(10)

Head $\rightarrow$ 10 $\rightarrow$ NULL
 $\downarrow$
current (End)

Step 2

After:

L1. append(20)

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ NULL
 $\downarrow$
current (End)

STEP 3

A.P.P.:

L1. append (5)

Head $\rightarrow 10 \rightarrow 20 \rightarrow 15 \rightarrow \text{NULL}$

1
current (end)

Final list (L1)

Head $\rightarrow 10 \rightarrow 20 \rightarrow 15 \rightarrow \text{NULL}$

1
Current

(b)

operations

L2. append (v);

L9. append (20);

L2. append(15);

l2. move to start U;

12. insert(3a);

L2. next() ;

L2-ingest (12) :

Step 1

After appending 10, 20, 15

Head $\rightarrow 10 \rightarrow 20 \rightarrow 15 \rightarrow \text{null}$

1
current (END)